W1 R DATA + COMPUTING

R objects: vector one type; matrix one type by coercion; data frame columns can differ but each column one type; factor stores integer codes + levels; list can mix arbitrary objects.

$length(df)$ gives columns; $nrow(df)$ rows; $dim(X)=(n,p)$; $X[2:4,]$ selects rows 2–4; $X[c(2,4)]$ on matrix is linear indexing.

Reproducible workflow: import → inspect `str/head/summary` → clean types/factors/missing → split train/test → fit/tune only on training → locked test evaluation → report code, seed, package versions.
EDA: plot response vs predictors, class balance, missingness map, pair plot, boxplot by class, residual plot, leverage/outliers. Visual claims need scale/axis/context.
Data leakage: preprocessing on full data, feature selection using all labels, SMOTE before split, tuning threshold on test, repeated test peeking. Fix: wrap preprocessing inside resampling folds.
Statistical computing: simulations need `set.seed`, vectorisation when possible, loops clear for MC logic. Store simulation outputs in vector, summarise by mean/quantile/SE.
Exam code diagnosis: check object class, dimension, training/test boundary, factor level order, response type, threshold direction, and whether random operations are seeded.

Object	Can mix types?	Common trap
vector	No	coerces char
matrix	No	linear index
data.frame	By column	length=cols
factor	Levels	numeric code trap

High-frequency recall: R type questions, confusion matrix calculations, CV fold size, model comparison wording, MC pseudo-code blanks.

W2 REGRESSION + SMOOTHING

Linear regression: $\hat{y}=\beta_0+\sum_j \beta_j x_j$; OLS minimises $RSS=\sum_i (y_i-\hat{y}_i)^2$. Interpret β_j as expected response change per unit x_j holding others fixed.

$R^2=1-RSS/TSS, 0\le R^2\le 1$ on training OLS w/intercept because RSS $\le$ TSS; adjusted $R^2=1-\frac{RSS/(n-p-1)}{TSS/(n-1)}$.

Likelihood: normal errors $Y_i\sim N(x_i^\top\beta,\sigma^2)$; negative log-likelihood $\frac{n}{2}\log(2\pi)+n\log\sigma+\frac{1}{2\sigma^2}\sum e_i^2, \sigma>0$. MLE $\hat{\beta}$ matches OLS under Gaussian errors.
Smoothing: estimate $m(x)=E(Y|X=x)$ without fixed linear form. Kernel smoother $\hat{m}(x_0)=\sum_i w_i(x_0)y_i/\sum_i w_i, w_i=K((x_i-x_0)/h)$.
Bandwidth h: small h low bias/high variance; large h high bias/low variance. KDE similar: $\hat{f}_h(x)=\frac{1}{nh}\sum_i K((x-X_i)/h)$.
Spline: piecewise polynomial joined smoothly at knots. More knots/lower penalty = flexible, risk overfit; smoothing parameter controls roughness penalty.

Method	Strength	Risk
linear	interpretable	miss nonlinearity
loess/local	flexible	boundary noise
spline	smooth	knot/penalty choice
tree	interactions	piecewise jumps

Model comparison: use validation/CV RMSE, MAE, residual plots, not training R^2 alone. If RMSE lower, model has smaller typical prediction error in response units.
Exam mini-proof: $RSS=\sum(y_i-\hat{y}_i)^2\ge 0$; OLS intercept model cannot fit worse than mean-only model, so $RSS\le TSS$ and $R^2\in[0,1]$ for training OLS.

W3 DENSITY + UNSUPERVISED

PDF: $f(x)\ge 0, \int f(x)dx=1$; probabilities are areas, not point heights. Histogram depends on bin width/origin; KDE smooths with kernel + bandwidth.

KDE: $\hat{f}_h(x)=\frac{1}{nh}\sum_{i=1}^n K\{(x-X_i)/h\}$; kernel integrates to 1, often symmetric. Small h overfits spikes; large h oversmooths modes.

MLE vs KDE: MLE assumes parametric family and estimates finite parameters; KDE is nonparametric and estimates shape directly. MLE efficient if model right, biased if family wrong; KDE flexible but bandwidth sensitive and suffers in high dimension.
Distance: Euclidean uses magnitude; Manhattan robust-ish to coordinate path; correlation distance ignores scale/offset pattern. Scaling essential when variables have different units.
K-means: minimise within-cluster sum squares; needs k, scaling, random starts; spherical/equal-size bias; sensitive to outliers.
Hierarchical: agglomerative merges by linkage. Single linkage = min pair distance, chaining; complete = max, compact clusters; average = mean pair distance; centroid may invert.

Dendrogram cut at height h: number of vertical branches crossed by horizontal line. Lower merge height = greater similarity under the chosen linkage.

Link	Cluster dist	Shape	Trap
single	min	chained	bridges
complete	max	compact	strict
average	mean	balanced	less interpretable
centroid	centres	fast	inversions

Cluster validation: no labels by default; use silhouette, gap statistic, stability, domain knowledge. Validation suggests, never proves, true k.
Scaling answer: yes if variables such as price, weight, engine size use different units; otherwise large-scale variables dominate Euclidean distance.

W4 HIGH-DIM VISUALISATION

PCA: orthogonal linear directions maximising variance. Compute on centred data; standardise if variables have different scales. Scores = projected observations; loadings = variable weights.

PC1 maximises $v^\top Sv$ s.t. $\|v\|=1$; variance explained by PC j is $\lambda_j/\sum_k \lambda_k$. Reconstruction with first r PCs gives best rank-r linear approximation under squared error.

PCA workflow: split first if used for modelling → fit centring/scaling/PCA on training → transform validation/test with same parameters. Never use test data to choose PCs.
t-SNE: nonlinear visualisation preserving local neighbourhood probabilities. Good for exploratory local clusters; bad for exact global distances, cluster sizes, axes interpretation, and reproducibility without seed/perplexity reporting.
PCA vs t-SNE: PCA linear, deterministic, interpretable loadings, global variance; t-SNE nonlinear, stochastic, local neighbourhoods, visual only. PCA can preprocess before t-SNE.
High-dimensional analytics: curse of dimensionality makes distances concentrate and overfitting easier. Use feature filtering, regularisation, PCA, embedded selection, domain constraints.

Task	Prefer	Reason
interpret variables	PCA	loadings
visual local groups	t-SNE	neighbourhood
predict w/ many p	lasso/ridge	controls variance
distance clustering	PCA/scale	noise reduction

PCA plot interpretation: mention which PCs, variance explained, group separation/overlap, variables driving PC1/PC2 by loading signs/magnitudes. Avoid causal claims.
Common PCA code bug: running `prcomp` before splitting or without `scale.=TRUE` when units differ; interpreting arbitrary sign of PCs as fixed.

W5 CLASSIFICATION CORE

Logistic regression: $p(x)=P(Y=1|x)=1/(1+e^{-(\beta_0+x^\top\beta)})$; log-odds linear. Threshold t controls class labels; coefficients affect odds ratios e^{β_j}.
LDA: assumes class conditional multivariate normal with shared covariance Σ; decision boundary linear. Better if Gaussian-ish classes with equal covariance; can be unstable if p high or covariance singular.
Logistic vs LDA: logistic directly models $P(Y|X)$, fewer distributional assumptions; LDA models $X|Y$ + priors, can be efficient when assumptions hold.
KNN: nonparametric, distance-based, flexible boundary; tune k, scale variables, prediction slow for large n. Small k high variance; large k high bias.
SVM: max-margin separator. Hard margin requires separability; soft margin minimises $\frac{1}{2}\|w\|^2+C\sum_i \xi_i$ s.t. $y_i(w^\top x_i+b)\ge 1-\xi_i$.

Large C penalises violations strongly: narrower margin, fewer training errors, more overfit risk. Small C: wider margin, more tolerance, possible underfit. Support vectors determine boundary.

Kernel: $K(x,z)=\phi(x)^\top\phi(z)$ gives nonlinear boundary without explicit features. RBF γ controls locality; large γ very wiggly.

Method	Boundary	Needs scaling?	Key tune
logit	linear	often	penalty
LDA	linear	yes-ish	priors
KNN	local nonlinear	yes	k
SVM	linear/kernel	yes	C,γ

Decision boundary from logit: if $p=0.5, \beta_0+\beta_1x_1+\beta_2x_2=0$; line slope $-\beta_1/\beta_2$, intercept $-\beta_0/\beta_2$.
Exam geometry: point class by sign of $w^\top x+b$; slack point violates margin if $y_i(w^\top x_i+b)<1$ and misclassified if <0.

W6 CV + BOOTSTRAP

Holdout: train fits parameters, validation tunes hyperparameters/threshold, test estimates final generalisation once.
k-fold CV: split training into k folds; each fold validation size n/k, training size $n(1-1/k)$ if equal. Stratify for imbalance.

5-fold with $n=200$: validation 40, training 160. 10-fold with $n=1000$: validation 100, training 900.

Nested CV: inner loop tune; outer loop estimate performance after tuning. Needed when reporting model-selection performance fairly.
Bootstrap: sample n observations with replacement from empirical distribution. About 63.2% unique observations appear in one bootstrap sample; out-of-bag about 36.8%.

Bootstrap SE: $\widehat{SE}(\hat{\theta})=sd(\hat{\theta}_1^*,\dots,\hat{\theta}_B^*)$; percentile CI uses empirical quantiles of bootstrap estimates.

Bootstrap use: uncertainty of complex statistic, bagging, random forest; not magic for dependent data unless block bootstrap used.
CV pipeline: for each fold: fit preprocessing on fold training only → train/tune candidate → predict fold validation → compute metric → average.

Goal	Method	Trap
tune model	CV	test peeking
uncertainty	bootstrap	iid assumption
small data	LOOCV/high k	high variance compute
time data	rolling split	random split leak

Recall trend: questions ask why validation is needed, what leakage changes, and how CV fold sizes are counted.

W7 MISSING + IMBALANCE

MCAR: missingness independent of observed/unobserved data. **MAR**: depends on observed variables. **MNAR**: depends on missing/unobserved value itself.

Complete-case unbiased only under strong conditions (often MCAR); mean imputation understates variance and distorts correlations; multiple imputation preserves uncertainty better.

Handling: delete if tiny MCAR; simple impute for prediction baseline; model-based/KNN/MICE for MAR; missingness indicator may help prediction; sensitivity analysis for MNAR.
Imputation pipeline: split first → learn imputer on training → apply to validation/test. For CV, impute inside each fold to avoid leakage.
Class imbalance: accuracy can be misleading; focus on metric tied to cost. False negative costly → sensitivity/recall; false positive costly → precision/specificity.

Confusion: Accuracy $=(TP+TN)/N$; Precision $=TP/(TP+FP)$; Recall/Sensitivity $=TP/(TP+FN)$; Specificity $=TN/(TN+FP)$; $F_1=2PR/(P+R)$.

Imbalance fixes: threshold moving, class weights, over/under-sampling, SMOTE, anomaly framing, PR-AUC. Apply sampling only within training folds.

Pred/Actual	+	-
+	TP	FP
-	FN	TN

Metric choice answer: name costly error → map to FP/FN → choose metric → explain trade-off.

Common bug: selecting high-correlation features on all data before split leaks labels; choosing threshold by test F1 invalidates test.

W8 FEATURE + MODEL SELECTION

Feature selection: reduce complexity/noise, improve generalisation/interpretability, lower compute. It does not transform to higher dimension; it may discard useful variables if unstable.
Filter: rank features without model, e.g. correlation, mutual information. Fast but ignores interactions. **Wrapper**: evaluate subsets with CV, e.g. forward/backward stepwise. Expensive, can overfit. **Embedded**: selection during fitting, e.g. lasso/tree importance.
Best subset: exhaustive over subsets, can find best training/CV subset but combinatorial. Forward stepwise greedy adds one at a time; backward removes; neither guarantees global best.

Ridge: minimise $RSS+\lambda\sum_j\beta_j^2$, shrinks but rarely zero. Lasso: $RSS+\lambda\sum_j|\beta_j|$, can set coefficients exactly zero.

λ: larger penalty → simpler model/more shrinkage/higher bias/lower variance. Choose by CV on training. One-SE rule chooses simpler model within 1 SE of minimum CV error.
High p, small n: use regularised linear model + CV; avoid unpruned trees/deep nets. Report selected features cautiously; selection uncertainty matters.
Information criteria: AIC $=-2\ell+2k$, BIC $=-2\ell+k\log n$; lower better, BIC penalises complexity more. Use only comparable likelihoods.

Method	Pros	Cons
filter	fast	ignores model
wrapper	task-aligned	expensive/leaky
lasso	sparse	tuning/correlation instability
ridge	stable	not selection

Exam phrase: feature selection must be learned inside resampling; otherwise validation error is optimistic.

W9 TREES + ENSEMBLES

Decision tree: recursively split feature space. Regression tree predicts leaf mean; classification tree predicts majority/probability. Greedy splits minimise RSS or impurity (Gini/cross-entropy).

Gini $=1-\sum_k p_k^2$; entropy $=-\sum_k p_k\log p_k$; regression split chooses largest RSS reduction.

Bias-variance: deep tree low bias/high variance; shallow tree high bias/low variance. Pruning/min leaf/depth controls overfit. Tree predictions are piecewise constant and unstable to small data changes.
Bagging: train trees on bootstrap samples and average/vote; reduces variance. **Random Forest**: bagging + random feature subset at each split; decorrelates trees; OOB samples estimate error.
Boosting: sequential trees fit residuals/gradients or reweight errors. Can reduce bias and variance but needs tuning; sensitive to noise/outliers.

Ensemble	Build	Main effect	Tuning
Bagging	parallel boot	variance	trees
RF	parallel+features	variance/corr	mtry,trees
GBM	sequential	bias+tune	depth,lr,trees

Tree prediction: follow splits from root; each branch condition determines next node; report leaf value and units.
RF vs GB answer: RF robust baseline, less tuning, averages decorrelated trees; GB sequentially improves current ensemble, often stronger but learning rate/depth/trees need tuning and overfit control.
ROC/AUC: threshold-free ranking summary for scores; AUC 0.5 random, closer 1 better. For severe imbalance, PR curve may be more informative.
Recall trend: compare tree vs linear regression with RMSE; explain poor tree via high bias piecewise constants or high variance unstable splits.

W10 MONTE CARLO I

Monte Carlo: approximate expectation/probability/quantile using repeated random simulation. If $X_1,\dots,X_M$ iid, $\bar{X}_M\to E[X]$ and $SE(\bar{X})=s/\sqrt{M}$.

MC error decreases at rate $1/\sqrt{M}$; 100x simulations gives about 10x smaller simulation SE. Report seed, M, estimator, uncertainty.

Generic pseudo-code: set seed; choose M; initialise output vector; for $i=1,\dots,M$ simulate process; store statistic T_i; estimate mean by `mean(T)` and 95th threshold by `quantile(T,.95)`.
Collector/coupon: repeated draws among n types until all collected. Completion time estimated by while loop `length(unique(collected)) < n`.

If each random arrangement succeeds with probability p, attempts $T\sim Geom(p)$ on $\{1,2,\dots\}$, $E[T]=1/p$, $Var(T)=(1-p)/p^2$. For perfect permutation of n labelled objects, $p=1/n!$.

Estimating a probability: $\hat{p}=M^{-1}\sum I(A_i)$; approximate $SE=\sqrt{\hat{p}(1-\hat{p})/M}$.
Quantile threshold: $t_{0.95}=\min\{t:P(T\le t)\ge 0.95\}$ estimated by empirical 95th percentile, not mean.

Need	Estimator	R code
mean steps	$\bar{T}$	mean(T)
95% guarantee	$q_{.95}$	quantile(T,.95)
prob event	$\bar{I}$	mean(event)
MC SE	$s/\sqrt{M}$	sd(T)/sqrt(M)

Exam code blanks: loop range, while condition, random draw, mean, quantile; explain M controls MC uncertainty not model truth.

W11 MONTE CARLO II + SAMPLING

Inverse transform: if $U\sim U(0,1)$ and $X=F^{-1}(U)$ then $X\sim F$. Example exponential $X=-\log(1-U)/\lambda$.
Acceptance-rejection: sample $Y\sim g, U\sim U(0,1)$; accept Y if $U\le f(Y)/(Mg(Y))$ where $Mg(x)\ge f(x)$ for all x. Acceptance rate roughly $1/M$ if f normalised.

Condition: find finite M s.t. $f(x)\le Mg(x)$ on support; proposal support must cover target support.

Importance sampling: $E_f[h(X)]=E_g[h(Y)f(Y)/g(Y)]$; choose g with heavier tails/large mass where hf important. Bad g gives huge weights/variance.
MCMC: construct Markov chain with stationary target distribution. Burn-in, mixing, autocorrelation and convergence diagnostics matter; effective sample size < raw iterations.
Metropolis-Hastings: propose $y\sim q(y|x)$; accept with $\alpha=\min\{1,\frac{\pi(y)q(x|y)}{\pi(x)q(y|x)}\}$. Symmetric proposal cancels q.
Gibbs: sample each full conditional in turn; special case of MH with acceptance 1 when full conditionals available.
Irreducible/aperiodic: chain must be able to reach all relevant states and not be trapped in deterministic cycles for convergence to stationary distribution.

Method	Use	Main risk
inverse CDF	easy F^{-1}	not available
accept-reject	known bound	low accept
importance	rare events	weight variance
MCMC;complex target	mixing/autocorr	

High-dim MC: plain MC rate independent of dimension for integration SE, but finding good proposals/covering important regions becomes hard; acceptance can collapse.

W12 INTEGRATED EXAM PATTERNS

Model-building audit: split data first; define target metric; impute/scale/select within training/CV; handle imbalance only inside training folds; tune with CV; lock model+threshold; evaluate once on test; report confusion metrics + uncertainty.
Confusion-matrix long answer: compute TP/TN/FP/FN carefully from actual vs predicted orientation; show denominator for each metric; interpret metric under class imbalance and cost.

$F_1=\frac{2TP}{2TP+FP+FN}$; balanced accuracy $=\frac{1}{2}(\text{sensitivity}+\text{specificity})$; false negative = actual positive predicted negative.

Diagnostic answer template: identify flawed step → state why it leaks/overfits/misaligns metric → correct step → consequence for validation estimate.
Common flaws: SMOTE before split; scaling on all data; feature selection before CV; using accuracy for rare positive class; minimising F1; treating CV as five deployable models; interpreting t-SNE distances globally.
Comparison answer: define both methods; state assumptions; explain bias/variance or interpretability; give situation where each wins. Use this for PCA vs t-SNE, logistic vs LDA, tree vs linear, RF vs boosting, MLE vs KDE.
SVM geometry answer: compute sign; identify support/slack via margin condition; discuss C effect; distinguish fitted-boundary prediction vs re-fitting effect.
MC answer: specify random experiment, stopping rule, statistic stored, estimator, quantile if threshold, seed and M, and MC uncertainty rate.

Recall signal	Likely topic	Best response
R object	type coercion	state class/dim
PCA bug	leak/scale	split+scale
spam matrix	imbalance	recall/F1
toy collector	MC	while+quantile
tree RMSE	bias-var	lower RMSE better

Exam discipline: answer the scenario, not just definitions. Always connect metric/model choice to data shape, assumptions, costs, and validation protocol.
Worked metric micro-example: actual spam 100, nonspam 900; predicted spam among actual spam 70, missed spam 30; false alarms 45. $TP=70, FN=30, FP=45, TN=855$. Accuracy $925/1000=.925$ can look high; recall $=.70$, precision $=70/115=.609$, $F_1=.651$. If missed spam costly, improve recall even if precision falls.
Worked CV micro-example: $n=120, k=5\Rightarrow 24$ validation + 96 training each fold. If tuning C,γ for SVM, scaling must be fitted on the 96 observations only; validation 24 are transformed using those fold means/SDs.
Worked PCA bug: `prcomp(all_x, scale.=FALSE)` before split has two flaws: all-data leakage and unit domination. Correct: split; in each training fold run `prcomp(train_x, center=TRUE, scale.=TRUE)`; predict scores for validation/test by same centre/scale/loadings.
Worked tree path: if split1 $area<90$ gives 640 else split2 $beds<4$ gives 860 else 1120, sample $area=115, beds=3$ goes right then left: 860. If CV RMSE tree 92 vs lm 70, lm better because RMSE lower.
Worked SVM: hyperplane $2x_1+3x_2+4x_3+1=0$. Point $(1,2,3)$: score $21>0$, positive. Margin violation if $y_if(x_i)<1$; misclassified if $y_if(x_i)<0$. Larger C punishes violations, often narrower margin.
Worked MC: for 20 toys, `while(length(unique(collected))<20)`; `newitem <- sample(1:20,1)`; output T_i=shops. Estimate $E[T]$ with `mean(T)`; 95% guarantee with `quantile(T,.95)`; MC SE $sd(T)/\sqrt{M}$.
Assumption checkpoints: LDA shared covariance + Gaussian-ish predictors; logistic no normality but linear logit; KNN/SVM distance methods need scaling; KDE needs bandwidth; bootstrap assumes iid unless block/bootstrap variant; k-means assumes meaningful Euclidean geometry.
When answers differ by cost: disease screening: FN severe ⇒ high recall/sensitivity; spam triage with costly inbox pollution: recall; fraud investigation with expensive manual review: precision; balanced benign classes: accuracy plus confusion matrix may be enough.
When answers differ by data size: small n, $p\gg n$: regularised linear/SVM, strict CV; huge n: KNN prediction costly, RF/GB feasible but tune; high-dimensional visualisation: PCA before distance/t-SNE; many missing values: imputation model + sensitivity.
When answers differ by interpretability: logistic/linear regression and PCA loadings are explainable; tree rules are simple but unstable; RF/GB often stronger but black-box; t-SNE is exploratory only; MC estimates are transparent if pseudo-code/stopping rule clear.
Answering compare questions: 1 definition each; 1 assumption each; 1 bias/variance or computation contrast; 1 scenario where A wins; 1 scenario where B wins; 1 validation check. This structure earns marks even when exact wording differs.
Answering code-fill questions: identify loop index i, state vector initialisation, stopping condition, random draw, statistic storage, final summary. Explain every blank using the simulation target, not just syntax.
Answering flaw-finding questions: mark each step as appropriate/inappropriate; for each inappropriate step give corrected order. Common correct order: split → training-fold impute/scale/sample/select → CV tune → validation threshold → final test.
Answering dendrogram questions: state linkage used; draw/cut horizontal line; count clusters; compare merge heights for similarity; scaling rationale; note complete vs single can alter tree because max vs min inter-cluster distance.
Answering model-selection questions: hyperparameters are chosen by validation/CV; test set only once. If using lasso/ridge, describe penalty and λ trend. If using AIC/BIC, lower is better but models must be comparable and diagnostics still matter.

Formula	Use	Denominator	Trap
Prec	flag quality	TP+FP	FP cost
Recall	capture positives	TP+FN	FN cost
Spec	capture negatives	TN+FP	not recall
F1	imbalance	$P+R$	needs class

Topic	Parameter	Too small	Too large
KDE	h	spiky	oversmooth
KNN	k	variance	bias
SVM	C	underfit/tolerant	overfit strict
Lasso	λ	complex	overshrunk
GBM	lr	slow	overfit

Model	Exam assumption line	Quick fix
LDA	equal covariance	regularise/QDA/logit
KNN	distance meaningful	scale/select features
PCA	linear variance	standardise/train only
RF	OOB internal check	still use test
MC iid sims	independent reps	seed+large M

Mini proof bank: $R^2\ge 0$ for train OLS w/intercept because mean-only model is feasible; lasso sparsity comes from L_1 corner geometry; bagging variance drops when tree errors are not perfectly correlated; RF random features lower correlation; boosting can chase residual structure sequentially.
Mini pseudo-code bank: bootstrap: for $b=1..B$, sample indices $1..n$ with replacement, compute statistic, store θ_b^*. CV: for fold j, train on $-j$, validate on j, average metric. MH: propose, compute ratio, accept/reject, store after burn-in.
R syntax reminders: `sample(1:20,1,replace=TRUE)`, `union(collected,newitem)`, `mean(pred==truth)`, `table(pred,truth)`, `quantile(T,.95)`, `sd(T)/sqrt(M)`.
Danger words: "guarantees best", "always", "scale invariant", "test tuning", "accuracy enough for imbalance", "t-SNE distance exact", "bootstrap fixes bias", "RF cannot overfit". These often mark false MCQ options.
If stuck: identify task type (metric/code/compare/audit/MC); write definitions + one scenario-specific reason; show denominator or stopping condition; explicitly state leakage/assumption if relevant. Partial marks come from structured reasoning.
One-line definitions: sensitivity=recall of positive; specificity=recall of negative; precision=purity of positive predictions; OOB=not in a tree bootstrap sample; support vector=point constraining margin; bandwidth=smoothing scale; perplexity=t-SNE neighbourhood scale.
Frequent short answers: RF less overfit than one tree because averaging decorrelated high-variance learners; GB differs by sequential residual/error focus; LDA can outperform logit if Gaussian equal-covariance assumptions hold; lasso selects, ridge stabilises.
Frequent calculations: CV fold counts use validation n/k; Euclidean distance requires equal vector length; F_1 undefined if precision/recall denominators zero unless convention stated; geometric attempts count includes the successful attempt.
Model warning signs: high train low validation performance = overfit; both poor = underfit/bad features; validation chosen many times = optimistic; many predictors and few rows = regularise; nonlinear boundary = KNN/SVM/RF/GB candidate.
Writing style: for 1 mark, give result + unit; for 2 marks, result + why; for 4–6 marks, compute/diagnose + interpretation + limitation/fix. Use numbers from the question and avoid generic textbook-only answers.

Cue	Likely method	Watch
labelled classes	classification	metric/cost
no labels	clustering/PCA	scale
density shape	KDE/MLE	bandwidth/model
uncertainty stat	bootstrap	iid
simulate process	MC	stopping

Edge-case bank: all predictions negative: positive recall 0; all probabilities tied: threshold/priority rule needed; zero variance predictor: cannot standardise normally; unseen factor level: prediction may fail; missing outcome: cannot train supervised label.
More edge cases: KDE near boundary can place mass outside support; KNN tie when even k; lasso with correlated predictors may pick one arbitrarily; PCA with uncentred data captures mean shift; t-SNE different seeds can alter layout.
Short derivations: $F_1=2PR/(P+R)=2TP/(2TP+FP+FN)$. Bootstrap unique rate $1-(1-1/n)^n\approx 1-e^{-1}=.632$. MC CI for $\hat{p}$: $\hat{p}\pm 1.96\sqrt{\hat{p}(1-\hat{p})/M}$.
If asked to justify scaling: distances, PCA, KNN, SVM, k-means, hierarchical clustering are scale-sensitive; tree splits are less scale-sensitive; linear/logistic coefficients change units but predictions can be same under consistent rescaling.
If asked to justify metric: write class prevalence first; name operational cost; compute at least recall/precision; explain why accuracy can be high under majority-class guessing; choose threshold using validation.
Past-paper pattern: many questions combine a simple calculation with an interpretation. Do the arithmetic, then add one sentence about direction: smaller RMSE better, higher recall catches more positives, larger bandwidth smoother, larger λ simpler.
Common R packages/functions: `glm(..., family=binomial)`, `lda`, `knn`, `svm`, `rpart`, `randomForest`, `trainControl(method="cv")`, `prcomp`. Know concept even if syntax varies.
Sanity checks: probabilities in $[0,1]$; [illegible]fusion counts sum to N; CV fold sizes sum to n; qu[illegible]d answers a probability guarantee, not an average; selec[illegible]unt $\le p$.
Final pass: show denominators, state assum[illegible]eakage, tie metric to cost, and distinguish prediction from inte[illegible]**t-minute priority**: know confusion metrics cold; recognise leak[illegible], λ, h, k as bias-variance knobs; know PCA/t-SNE limits, write MC pseudo-code with mean vs quantile distinction.

USYD各科往届题库